The Internet of Things (IoT)

Competency Level 11.2: Explores the Internet of Things (IoT) to create a simple application

Learning Outcomes:

- Defines IoT (Internet of Thing)
- Identifies the needs of IoT to make day to day life smart
- Discusses the various applications of IoT
- Identifies the enabling technologies for IoT
- Designs and implements an IoT application to remotely control a device over Internet Example: switching a LED on and off over the Internet
- Uses IoT based systems while knowing the social and security consequences of IoT

Smart World

IoT can create a "Smart World" where life is full of autonomous interconnected smart systems. For example, what if your smart alarm clock not only wakes up you in the morning but also notifies your electric kettle to boil water for your morning tea? How about smart refrigerator monitors the level of food items in it and send you a shopping list to your smartphone in the weekend? The whole idea is to make our life more convenient and comfortable.

Things are everyday objects from small wristwatches to large cars and buildings. When added computing power, things become smart and are called embedded systems. Embedded systems can sense the state of the physical world and change the state of the physical world. When we connect the embedded systems to the Internet, they start interacting with themselves and users and create an Internet of Things.

Enabling Technologies

Many technologies have enabled the Internet of Things. They include the Internet Protocol (IPv6) with an extremely large address space, increasing bandwidth and decreasing the cost of networking technology, increasing functionality and decreasing size and cost of sensor technology and increasing performance and capacity and decreasing power consumption, size and cost of processor and storage technologies, etc.

Applications of IoT

IoT can be applied to any physical system such as wearables, appliances, homes, transport, and agriculture, health, etc. Consumer IoT applications includes mart wearables with monitoring and home appliances with remote accessing capabilities. Commercial IoT applications include smart medical and health care systems with remote monitoring, emergency notification and assistive capabilities, smart building and home automation applications with capabilities to monitor and control lighting, climate, entertainment, security and other systems in a building, transportation applications such as smart traffic lights, smart parking,vehicle control, and safety and roadside assistance systems etc. Industrial IoT applications include smart manufacturing applications such as process controls and predictive maintenance systems and agriculture applications such as rainfall, humidity, pest infestation, and soil content monitoring systems and smart irrigation and fertilization systems, etc.

Challenges of IoT

IoT opens the door to a lot of possibilities and opportunities but also to many challenges. They include social concerns such as social isolation and security and privacy concerns such as unauthorized control of IoT devices and access to personal and sensitive information. What if someone hacks into your smart alarm clock and then gets access to your entire network of things? Therefore, it is very important to understand not just the opportunities but also the challenges of IoT.

IoT Based System Development

IoT based systems are developed to automate the intercommunication of smart systems.

SYSTEM5: Smart Light

This system explains how to turn on and off a light remotely over the Internet. When HTTP requests *http://host/?1* and *http://host/?0* are made using a web browser running on remote computer connected to the Internet, the Smart Light would turn on and off an LED, respectively.

Required components

1 × Arduino Uno microcontroller-based development board
(to control smart light system)
1 × ArduinoEthernet Shield
(to extend the functionality of Arduino Uno to connect to the Internet)
1 × LED (to emit the light)
1 × 220Ω Resistor (to drop the voltage and limit current flow to the LED as required)

Arduino Ethernet Shield

Arduino Ethernet Shield is a pre-assembled add-on module to the main microcontroller-based development board. Once added to the Arduino Uno board, the Ethernet Shield extends the functionality of Arduino Uno and facilitates it to connect to the Internet. Figure25 shows the latest version of Arduino Ethernet Shield.

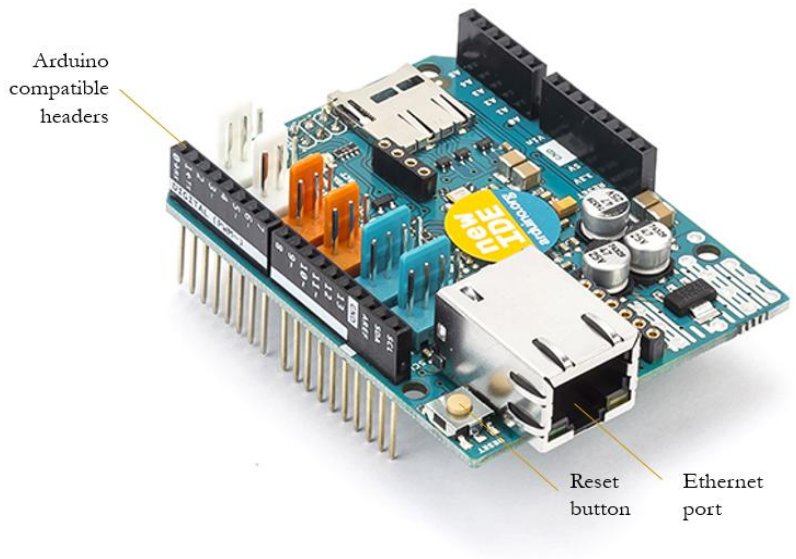


Figure 29: Arduino Ethernet Shield

Construct Schematic Diagram and Assemble Hardware

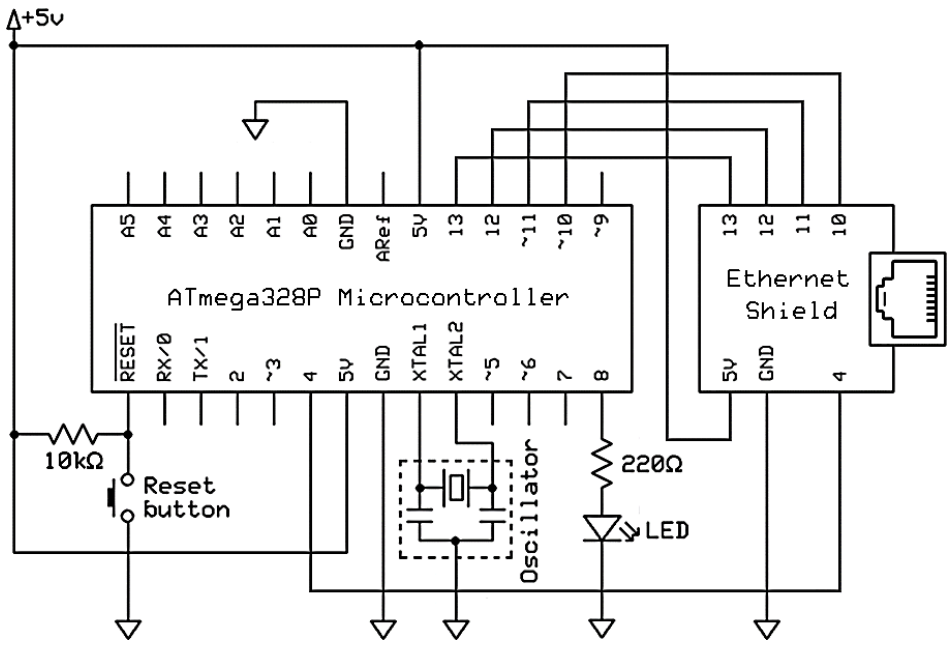


Figure 30: Schematic diagram of Smart Light

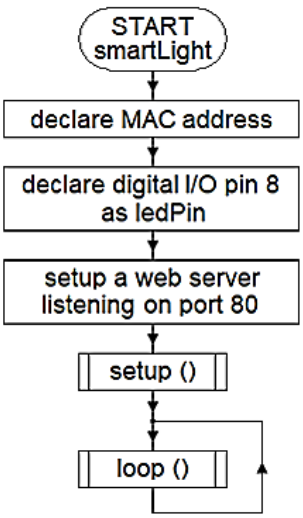
Figure 30 depicts the schematic diagram of Smart Light system. As in the previous systems, Arduino Uno comes with most of the required components pre-connected. The additional components to be connected are the Ethernet Shield and an LED to be controlled over the Internet with a resistor to a digital I/O pin. When stacked on Arduino Uno, the Ethernet Shield communicates with the microcontroller in the Arduino Uno board using the digital I/O pins 4, 10, 11, 12 and 13. Therefore, they cannot be used as general-purpose I/O pins.

Design Firmware

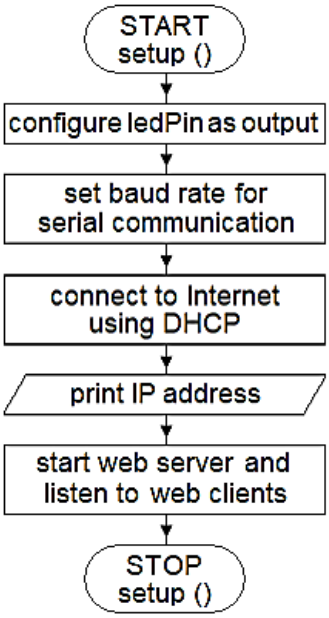
As in the previous systems, the firmware development starts with designing an algorithm using a flowchart.

Flowchart

Figure 31 shows the flowcharts for the Smart Light.



The flow chart of setup( ) function is shown below:



The flow chart of loop( ) function is shown below:

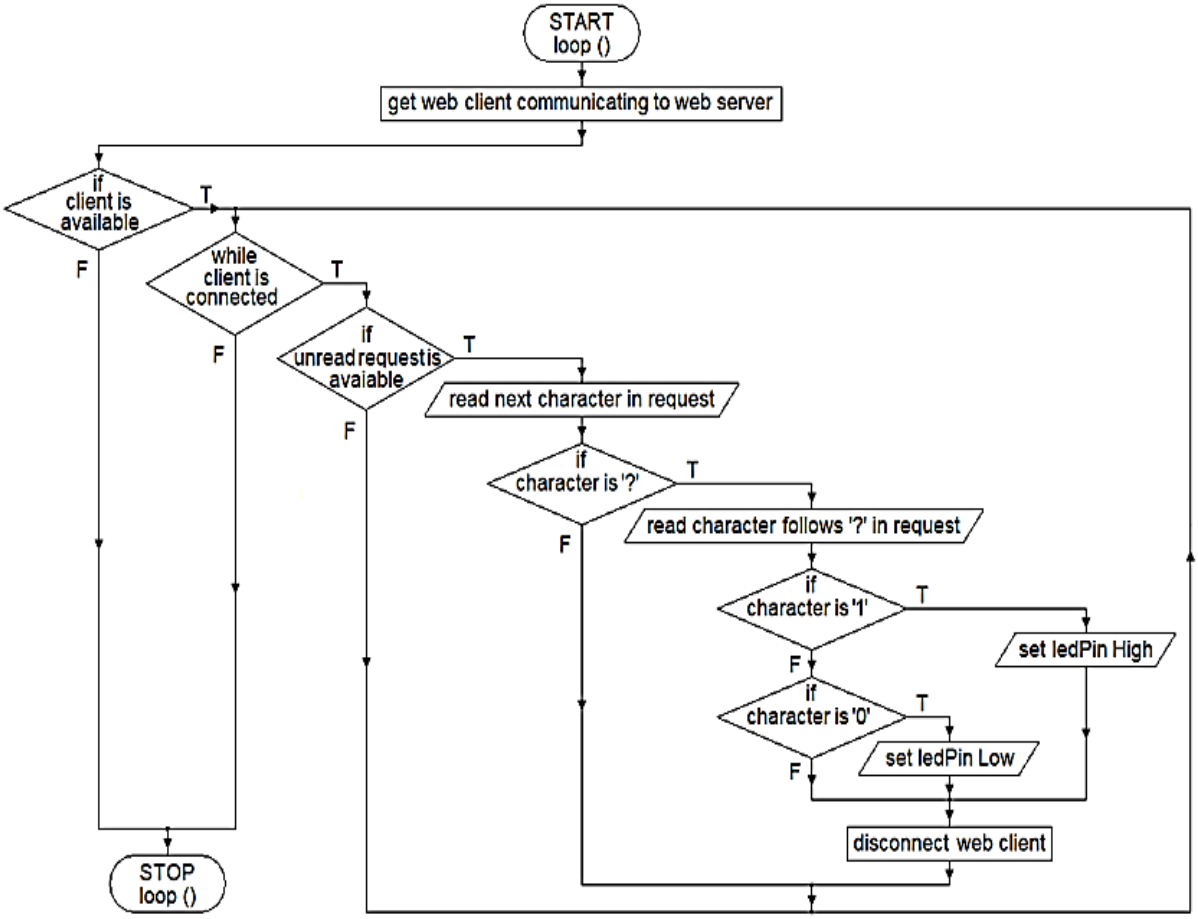


Figure 31: Flowchart of Smart Light

In the above algorithm, first, the hardware MAC address and digital I/O pin 8 are declared. Then a webserver is setup for listing any web client communication over port 80. In the setup() function, first the ledPin is configured as an output. Then the data communication rate for serial communication between the Arduino Uno board and the connected computer is set. This is followed by an attempt to connect to the Internet using DHCP. In the next step of setup() function, the IP address assigned by DHCP is printed on the serial monitor of Arduino IDE running in the computer connected. Then the previously setup webserver is started for listening to any incoming communication from web clients.

In each iteration of the loop() function, first it gets any web client communicating to the webserver. If such a web client is available, a loop is executed while the web client is connected to the webserver. In the body of this loop, it checks whether any unread HTTP request is available from the web client. If so, the request is read character by character. If the character is '?', then it checks whether the next character in the request is '1' or '0'. If so, the logic state of ledPin is set either high or low and the connected LED is turned on or off, respectively. Finally, the web client is disconnected from the webserver.

Develop Firmware

Figures 32 and 33 show the source code of firmware written based on the above flowchart using the Arduino IDE.

```
// turns on and off an LED over the Internet

#include <Ethernet2.h>

byte mac[] = {0x##, 0x##, 0x##, 0x##, 0x##, 0x##};
const int ledPin = 8;
EthernetServer webServer = EthernetServer(80);

void setup()
{
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
  Ethernet.begin(mac);
  Serial.println(Ethernet.localIP());
  webServer.begin();
}
```

Figure 32: Source code of Smart Light - Part 1

#include<Ethernet2.h> directive adds the *Ethernet2.h* library file required to work with Arduino Ethernet Shield 2 to the source code. The *mac[]* is an byte array containing the MAC address of the Ethernet Shield. Its *0x##* should be replaced with the hexadecimal numbers of the actual MAC address that comes with Ethernet Shield. The next line setups a webserver for listing any web client communication over port 80.

In the setup() function, *Serial.begin(baudRate)* function sets the data communication rate for serial communication between the Arduino Uno board and the connected computer. *Ethernet.begin(mac)* function connects the Ethernet Shield to the Internet using DHCP. In the next line in source code, *Ethernet.localIP()* function gets the dynamic IP address assigned by DHCP and *Serial.println(value)*function prints it on the serial monitor of Arduino IDE. The *begin()* function of the previously setup webserver starts the webserver and begins listening to any incoming communication from web clients.

```
void loop()
{
  EthernetClient webClient = webServer.available();
  if (webClient)
  {
    while(webClient.connected())
    {
      if(webClient.available())
      {
        char character = webClient.read();
        Serial.print(character);
        if (character == '?')
        {
          character = webClient.read();
          Serial.println(character);
          if (character == '1')
            digitalWrite(ledPin, HIGH);
          else if (character == '0')
            digitalWrite(ledPin, LOW);
          webClient.stop();
        }
      }
    }
  }
}
```

Figure 33: Source code of Smart Light - Part 2

In each iteration of the loop() function, the *available()* function of the webserver returns any web client communicating to the webserver. If such a web client is available, then a while loop is executed till the *connected()* function of web client returns false. In the body of this loop, *available()* function of web client checks whether any unread HTTP request is available from the web client. If so, *read()* function of web client reads next character in the request to a character variable. If the character is '?', then *read()* function of web client reads the character that follows '?' in the request to the character variable. The next two conditions check whether this character is '1' or '0'. If so, *digitalWrite(pin, state)* function sets the state of specified digital I/O pin logic HIGH or LOW and turns on or off the connected LED, respectively. Finally, the *stop()* function of web client disconnects the web client from the webserver.

Compile firmware and Uploade machine code

Now the Smart Light system requires to be connected to a network with a connection to the Internet using a network cable and the Ethernet port of the Ethernet Shield. As in the previous systems, then compiling firmware and uploading the machine code are done. Once uploaded the machine code, it requires open the serial monitor of Arduino IDE and gets printed the IP address of the webserver running. When HTTP requests ***http://host/?1*** and ***http://host/?0*** are made using a web browser running on another computer connected to the Internet, the Smart Light would turn on and off the LED, respectively. In the above URLs, the host means the IP address of the webserver running on the Smart Light system. Further,IoT based systems can also be developed using WiFi wireless communication as well.

Review Questions

1. What is an Embedded System?
2. Like general-purpose computer systems, do embedded systems also follow Input, Process and Output (IPO) model? Explain using an example.
3. Consider the following statement, "Embedded systems perform physical computing." Explain using an example.
4. Discuss the pros and cons of microprocessors-based and microcontrollers-based embedded systems developments.
5. Why do we need to code an infinite loop when writing firmware for microcontrollers? Explain.
6. What is the Internet of Things (IoT)?
7. How does IPv6 affect the development and implementation of the IoT?
8. Discuss the impact of IoT on our lives.